SFS Lab: Writing a Multithreaded File System

15-213/15-513: Introduction to Computer Systems — self-study edition

Last updated: July 4, 2026

This is an independent self-study adaptation of the Shark File System (SFS) lab from CMU 15-213/15-513 (Summer 2024). All original course materials, including the SFS code base and the lab design, are the intellectual property of Carnegie Mellon University and the 15-213/15-513 course staff. This adaptation is a personal self-study project; no affiliation with CMU is implied, and nothing here is an official release. Do not publish completed solutions.

1 Introduction

A file system is a data structure that the operating system uses to store, maintain, and refer to meaningful data on the disk, and conversely to derive usable information from the raw data on the disk. File systems allow programs to interact with data as abstract files, leaving the cumbersome work of putting that data onto the physical disk to lower-level calls. File systems are also simple but powerful tools for organization (with folders, linking, etc.) and security (with permissions).

In this lab, we provide you with a very basic file system, which does not implement folders, linking, or permissions. In the Shark File System, as distributed in this lab, you may only create and delete files, and read or write to them.

The lab has two parts. The first part deals with file system capabilities: you will implement a few functions that introduce new abilities of the file system, allowing users to see and change the position of file descriptors, and to rename files with a single call.

The second part asks you to improve the file system's safety and performance. Currently, there is nothing to prevent two simultaneous calls to the file system from overwriting or otherwise conflicting with each other. You will prevent conflicts like these from happening, while still allowing concurrent and fast access to the file system for separate users.

2 Logistics

This edition of the lab is self-contained and self-graded: there is no Autolab, no handin directory, and no course server. The test driver described in Section 6 plays the role of the autograder, and the score it prints is the score the lab gives you.

The only file you need to modify is `sfs-disk.c`. You must not modify `sfs-api.h` (the specification you are implementing), and you should not need to modify any other file. Some ambitious designs for the optional challenges mentioned in the `sfs-disk.c` comments may require changes to `sfs-disk.h`; if you change that file, you must update `sfs-fsck.c` to match.

The lab is intended for a 64-bit GNU/Linux environment (a native installation, WSL2, or a Linux container) with `gcc`, `make`, and POSIX threads.

3 Hand Out Instructions

Start by extracting the handout tarball in a directory where you plan to do your work:

```
linux> tar xf sfslab-handout.tar
linux> cd sfslab
linux> make
```

The important files are:

`sfs-disk.c`
: The SFS implementation, and the only file you need to edit. Three functions in it are unimplemented, and the whole file is thread-unsafe until you make it otherwise.

`sfs-api.h`
: The API specification. The doc comment above each function tells you exactly what that function must do. Read it carefully before writing any code.

`sfs-disk.h`
: The on-disk data structures and constants. Worth reading: it defines the high-level design of the file system.

`sfs-support.c`
: Low-level disk image and mmap support routines. Provided; do not modify.

`test-sfs.c`
: The test driver and autograder (Section 6).

`sfs-fsck.c`
: A file system consistency checker (Section 7.2).

`sfs-baseline-ref.c`
: The handout implementation wrapped in a single global mutex. Used by `make baseline` to calibrate the performance score for your machine. Do not modify.

Typing `make` produces three executables: `test-sfs` (the driver), `sfs-fsck` (the checker), and `test-sfs-baseline` (the calibration reference). Before you implement anything, run `./test-sfs` once: traces A00, A01, B00, and all of Category C should already pass, and the traces that exercise `sfs_getpos`, `sfs_seek`, and `sfs_rename` should fail. That is the expected starting state, and your job is to make the rest of the scoreboard green.

4 The SFS File System

4.1 The on-disk format

An SFS disk image is an ordinary file, mapped into memory with `mmap`(2) and divided into 512-byte blocks. Block 0 is the *superblock*; it identifies the image (a magic number), records the total number of blocks, and embeds the root directory: an array of 15 fixed-size directory entries, each holding a file name (at most 23 characters plus a terminating NUL), the file's size in bytes, and the block number of the file's first block. A directory entry whose first-block field is zero is unused.

Every other block begins with a 12-byte header containing a type tag and the block numbers of the previous and next blocks in whatever chain the block belongs to. The remaining 500 bytes of each block hold file data. Blocks form doubly linked lists: every file is a chain of blocks, and all unallocated blocks sit on a free list rooted in the superblock. Block number 0 acts as the NULL pointer, so the superblock can never appear in the middle of a chain. This is the same flavor of design as the classic FAT file system, except that the links live inside the blocks themselves.

A file always owns at least one block, even when it is empty: a nonzero first-block field is what marks a directory entry as live. A file of size 0 to 500 bytes occupies one block, 501 to 1000 bytes two blocks, and so on. The end of a file is known both from the size field in its directory entry and from the last block's next pointer being 0.

4.2 The provided API

The public entry points, declared and specified in `sfs-api.h`, are:

`sfs_format(diskName, diskSize)`
: Create and format a fresh disk image, then make it the active image.

`sfs_mount(diskName)` / `sfs_unmount()`
: Load an existing image / write out and deactivate the current one. Unmounting fails with `-EBUSY` if any files are open.

`sfs_open(fileName)`
: Open a file, creating it if necessary; returns a small nonnegative "file descriptor" usable only with these routines.

`sfs_close(fd)`
: Close a descriptor. Never fails.

`sfs_read(fd, buf, len)` / `sfs_write(fd, buf, len)`
: Read or write up to `len` bytes at the descriptor's current position, advancing the position. Writes extend the file (and allocate blocks) as needed.

`sfs_remove(name)`
: Delete a file. Deleting an open file fails with `-EBUSY`.

`sfs_list(cookie, filename_out, filename_space)`
: Iterate over the directory, returning one file name per call.

All functions return 0 or a nonnegative result on success and a negative errno value on failure. The doc comments in `sfs-api.h` are the authoritative specification, including the exact error codes the driver checks for.

4.3 The functions you must implement

Three functions in `sfs-disk.c` currently return `-ENOSYS`. Implementing them is your first order of business.

`ssize_t sfs_getpos(int fd)`
: Return the current position of descriptor `fd`, in bytes from the start of the file. If `fd` is not a valid descriptor, return `-EBADF`; this is the only possible error. This function is a warm-up to get you reading the descriptor bookkeeping in `sfs-disk.c`.

`ssize_t sfs_seek(int fd, ssize_t delta)`
: Move the position of `fd` by `delta` bytes, forward or backward, and return the new position. If the motion would make the position negative, clamp it to 0; if it would move past the end of the file, clamp it to the file size. (This clamping is deliberately different from `lseek`(2), to make your life simpler.) Note that a position is not just an integer: the descriptor also caches which block the position lies in, so seeking across block boundaries means walking the file's block chain. Given the output `loc` from an earlier `sfs_getpos` call, `sfs_seek(fd, loc - sfs_getpos(fd))` must return the descriptor to the earlier position.

`int sfs_rename(const char *old_name, const char *new_name)`
Rename a file. If `new_name` already exists, atomically delete and replace it: there must be no window in which a concurrent thread can observe `new_name` not existing. For the single-threaded version this is straightforward; once you add locking, keeping this promise is part of the design work.

5 Making the File System Concurrent

Once the three functions above work, the second half of the lab is to make the file system safe and fast under concurrent use. We suggest approaching this in three gradations.

First, make the system thread-safe in a very coarse way, ensuring correctness only. This can be done by locking the entire file system around every API call, exactly as the calibration reference `sfs_baseline_ref.c` does. Expect a performance ratio near 1.0× and a performance score of 3/10 (Section 6.3). That is the intended first checkpoint, not the destination.

Second, think about readers and writers. When reading, no shared data changes except the reading descriptor's own position, so multiple readers of the same file can proceed together, whereas writing wants exclusivity. Be careful about where a reader/writer lock actually helps: because `sfs_read` advances the descriptor's position, a single global rwlock buys almost nothing — nearly every operation still needs the write side. Reader/writer locks pay off for structures that are genuinely read-mostly, such as the directory during name lookups, usually in combination with the per-file locks below.

Third, exploit separate files. Operations on different files touch disjoint data (apart from the shared allocator and tables), so reading or writing one file should not block work on another. A lock per directory slot (or per open file), plus short critical sections on the genuinely shared structures — the free list, the directory, and the open-file tables — is the level of granularity the performance score is calibrated around.

Whatever design you choose, fix a global lock *order* and stick to it everywhere; two threads that take the same locks in opposite orders will deadlock. The driver's 30-second per-trace timeout will catch a deadlock, but a design that cannot deadlock is far more pleasant to debug.

6 Evaluation

Your score is computed out of 26 points:

Category	Points	What it tests
A: feature traces (A00–A04)	5	format/mount, open/close/read/write, getpos, seek, rename
B: sequential traces (B00–B03)	4	remove/list and slot reuse, multi-block seek, overwrite-after-seek data integrity, error codes and edge cases (including full-disk `-ENOSPC` and descriptor-table `-EMFILE`), rename/list interactions
C: concurrent traces (C00–C02)	3	concurrent writes to separate files, concurrent reads of a shared file, mixed read/write, an open/close storm, and directory churn (create/rename/remove) under a live listing
Performance	10	concurrent throughput against a calibrated baseline
Style	4	manual self-review (not auto-graded)

6.1 Correctness (12 points)

Each trace is worth one point; the driver prints a PASS/FAIL line per trace followed by the details of any failed checks. After every trace unmounts its disk image, the driver also runs `sfs-fsck` on the image: a trace fails if the visible API results look right but the on-disk structure is corrupt (orphaned blocks, inconsistent links, sizes that disagree with the allocation). The first checker diagnostic is included in the failure output.

The Category C traces run each SFS call under a serializing wrapper, so they measure functional correctness under interleaved — but not truly simultaneous — calls. This keeps the feedback stable while your locking is unfinished. C02 ends with a directory-churn phase: four threads create, rename, and remove their own files while a fifth repeatedly lists the directory. While `sfs_rename` is still the `-ENOSYS` starter stub, the churn threads fall back to plain removes, so Category C passes on the unmodified handout; once you implement `sfs_rename`, the same phase exercises overwrite-rename against the live listing.

Every trace — A and B included — runs in a forked child with a 30-second wall-clock budget. A deadlock, crash, or assertion failure in your code therefore fails that one trace (reported as TIMEOUT or CRASH) instead of taking down the driver.

6.2 Race detection

After all twelve correctness traces pass, the driver compiles a ThreadSanitizer build of your implementation and re-runs the Category C traces with the serializing wrapper *disabled*, making real concurrent calls. The rerun sweeps three seeded "schedule-fuzz" runs, each injecting a different pattern of random yields and microsleeps around SFS calls, so races that hide under one particular interleaving still get a chance to surface. Because the sweep reruns the same C traces, the C02 churn phase hands the sanitizer real concurrent create, rename, remove, and list executions — forgetting to lock `sfs_rename` is caught here even though the serialized run cannot see it. If TSan reports a data race — or the sanitized traces fail or time out — the Category C score is set to 0, even though the normal traces passed. The failing seed is reported so you can replay it (Section 7.3).

If your system cannot run ThreadSanitizer at all, the check is skipped without penalty — but understand what that means: the serialized C traces alone cannot see data races, so on such a machine the Category C score is a functional signal only. The driver says so when it skips; rerun on a TSan-capable machine before trusting the concurrency score.

6.3 Performance (10 points)

The performance benchmark runs only after correctness is 12/12. It starts 8 threads, each working on its own file. A thread performs 2000 *sessions*; one session is one `sfs_open`, then 10 rounds of write/seek/getpos/read, then one `sfs_close` — 672,000 API calls per run in total. Timing starts at a barrier after all threads exist, so thread startup is excluded; the benchmark is sampled 5 times and the median ops/sec is scored.

The workload validates its own results as it runs: every return value is checked against the position the descriptor must be at, and each session's first read is compared against the bytes just written. If any check fails, the performance score is 0 — an implementation that is fast but wrong on the benchmark path earns nothing.

Raw throughput is machine-dependent, so the score is a ratio against the provided baseline (the handout implementation under one global mutex), measured on *your* machine by `make baseline`:

Ratio	Score	Interpretation
≥ 2.50	10/10	scaling that requires fine-grained locking
≥ 1.80	9/10	strong parallelism
≥ 1.40	7/10	meaningful parallelism
≥ 1.20	5/10	measurably better than one global lock
≥ 0.85	3/10	coarse-lock territory (this *is* the baseline)
< 0.85	0/10	slower than the naive baseline

These bars are calibrated against real implementations: a correct solution that keeps one global mutex measures about 1.0× and earns 3/10, while a per-file locking scheme measures 4.5–5× on an 8-hardware-thread machine and clears 10/10 with a wide margin. The 2.5× bar is deliberately below what per-file locking achieves even on a 4-core machine, so full marks stay reachable on small CPUs.

If `.perf_baseline` is missing, or was calibrated with a different benchmark version (the file carries a `WORKLOAD=` tag), the driver falls back to absolute throughput *capped at 5/10* and tells you to re-run `make baseline`. Full performance credit always requires a calibrated ratio.

6.4 Style (4 points)

There is no style autograder in this edition; the 4 style points are a self-review. Hold yourself to the standard of the course: consistent formatting (a `.clang-format` matching the handout style is included), no dead code or stray debug output, comments that explain *why* rather than what, error paths that clean up after themselves, and locking whose correctness a reader can check locally instead of by global reasoning.

7 Testing Your Work

7.1 The test driver

Run everything from the handout directory:

```
linux> ./test-sfs
```

The driver prints the scoreboard described in Section 6. Two flags control verbosity: `-q` suppresses per-check FAIL detail (scoreboard only), and `-v` raises the per-trace detail cap (by default only the first 3 failed checks per trace are printed). The exit status is the correctness gate: 0 once all twelve correctness traces pass, 1 otherwise. The performance score never affects the exit status; read it from the scoreboard.

When a trace fails, the driver saves a copy of the trace's surviving disk image(s) under `fail_<trace>_<image>.img` and prints the path, so you can inspect the exact corrupted state instead of re-running under a debugger. `make clean` removes the copies, or set `SFS_KEEP_FAILED_DISKS=0` to disable them.

7.2 The consistency checker

`sfs-fsck` verifies the structural integrity of a disk image: mislabeled blocks, invalid directory entries, file sizes that disagree with the number of allocated blocks, inconsistent or circular block chains, blocks on more than one chain, and orphaned blocks on no chain at all.

The driver names its working images `test.img` (Categories A and B), `test_conc_*.img` (Category C), and `test_perf.img` (the benchmark), and removes them again by the end of a run. The image you point the checker at is therefore usually a `fail_*.img` snapshot from Section 7.1 — or any im-

age your own experiments create with `sfs_format`:

```
linux> ./sfs-fsck fail_B01_test.img            # any structural errors?
linux> ./sfs-fsck -v fail_B01_test.img         # narrate the check
linux> ./sfs-fsck --dump fail_B01_test.img     # show the structure found
```

`--dump` prints what the image actually contains — superblock summary, free list, every directory entry with its block chain, and a block-type tally — before judging it. The dump is deliberately defensive: chains are walked with a step cap and out-of-range block numbers are annotated rather than crashing, so it stays useful on precisely the corrupted images you will want to point it at.

7.3 Race detection by hand

The driver runs TSan for you, but reruns with full stack traces are often more useful when hunting a specific race:

```
linux> gcc -std=c11 -g -fsanitize=thread -pthread -D_GNU_SOURCE=1 \
           -I. -o test-sfs-tsan test-sfs.c sfs-disk.c sfs-support.c
linux> ./test-sfs-tsan --tsan-only --sched-fuzz=2
```

Replace `2` with whatever seed the driver reported. The same seed reproduces the same schedule perturbation, so a failure found under fuzz is replayable. The scored run never fuzzes; only the TSan sweep and explicit `--sched-fuzz` invocations do.

7.4 Calibrating and reading the performance score

Calibrate once per machine (and again if the machine's configuration changes):

```
linux> make baseline        # writes .perf_baseline (median of 5 runs)
linux> ./test-sfs           # scored run, prints the ratio
```

`make grade` does both in one step. The calibration prints a `spread` line; if min and max differ by more than about 10%, the machine was not quiet — close the browser and re-run. Use `make baseline BASELINE_RUNS=11` for a tighter (odd) sample count on noisy machines.

7.5 Notes for Docker and WSL users

If your working directory is bind-mounted from a Windows host (the Docker Desktop default), every disk-image operation crosses a slow translation layer, and single-run variance of 40–70% on the benchmark is normal. The 5-sample median absorbs most of it, but if the driver warns that the spread exceeds 20%, point the disk images at the container's native file system:

```
linux> export SFS_DISK_DIR=/tmp
linux> make baseline        # recalibrate in the new location
linux> ./test-sfs
```

The chosen directory is recorded in `.perf_baseline`; if the calibration and the scored run disagree about `SFS_DISK_DIR`, the driver prints a loud warning, because the two measurements would be of different file systems. Moving the whole project into the Linux-native file system (e.g. under `\\wsl$\Ubuntu\home`) is the most robust fix.

8 Hints

- Implement `sfs_getpos` first — it is nearly free once you find the right field — then `sfs_seek`, then `sfs_rename`. Re-run `./test-sfs` after each one.
- Read the block-walking loops in `sfs_read` and `sfs_write` before writing `sfs_seek`; your seek must maintain the same descriptor invariants those loops rely on (position and cached current block agreeing with each other).
- Start the concurrency work coarse, verify correctness, then refine. A correct slow file system outscores a fast corrupt one by a mile: the performance benchmark does not even run until correctness is 12/12.
- Run `sfs-fsck` after concurrent tests. Races often corrupt the block chains in ways that reads happen not to notice; the checker notices.
- Be careful with the operations that touch global state: the free list, the directory, and the open-file tables. Each can have its own small lock, but fix a lock order.
- The comments in `sfs-disk.c` mention several optional design challenges (zero-block empty files, growing the directory past 15 entries, Unix-style delete-while-open). They are not graded; they are there if you finish early and want to keep going.

Appendix: Frequently Asked Questions

"No space left on device"
: Each test sizes its disk image appropriately for correct execution. If your implementation leaks blocks or double-allocates under a race, an operation can fail with `-ENOSPC` even though a correct implementation would have room. Run `sfs-fsck` on the failing image; leaked blocks show up as "not on any block list."

How do I debug disk corruption?
: `./sfs-fsck -v` narrates the whole check, and `--dump` shows the structure it found. The `fail_*.img` snapshots the driver keeps for failing traces are exactly the images to inspect.

What if `sfs_seek` goes past the end of the file?
: Clamp to the file size; if it would go negative, clamp to 0. This is specified in `sfs-api.h` and is deliberately different from `lseek`(2).

Does `sfs_rename` need to be atomic?
: Yes. If `new_name` already exists, the replacement must leave no window in which a concurrent thread can observe the name missing. For the single-threaded milestone this falls out naturally; once you add locking, make sure no code path briefly deletes the destination before the source takes its place.